AMRITA VISHWA VIDYAPEETHAM
UNIVERSITY
Established u/s 3 of UGC Act 1956

19ECE204/19CSE204
Digital Electronics and Systems
III Semester
B. Tech. (ECE & CSE)
Amrita School of Engineering, Bengaluru
Lecture 9 to 12

4.2 Strategy of Minimization

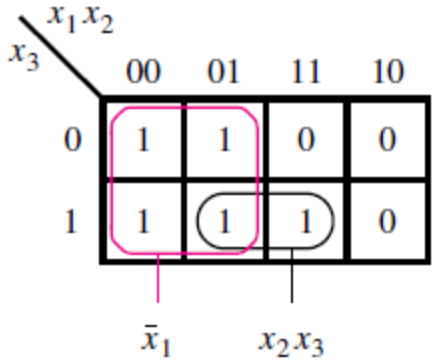
- Requirement:
  - Find few number of groups as possible
  - Find groups as large as possible groups of 1s (or 0s) that cover all cases where function has a value of 1 (or 0) -> results in fewer number of input variables in the corresponding product term (or sum term)
- Why Strategy of Minimization?
  - When many input variables are there in a function -> minimum cost implementation need to be derived

Terminology

- **Literal: Each appearance of a variable, either uncomplemented or complemented form is called a Literal**
  - Example: x1’x2’x3x4 has 4 literals
- **Implicant: A product term that indicates the input the**

input valuation for which a given function is equal to 1 is called an implicant of the function. The most basic implicants are either minterms for Product terms (and maxterms in case of sum terms). For an n-variable function, a minterm is an implicant that consist of n literals.

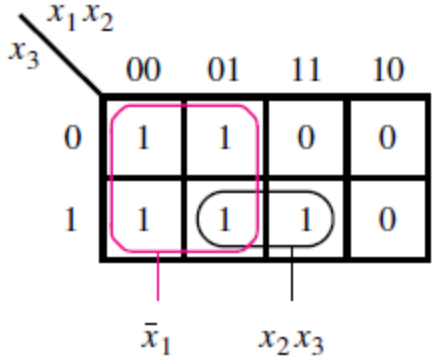
  - Example: five minterms: *x*1’*x*2’*x*3’, *x*1’*x*2’*x*3, *x*1’*x*2*x*3’, *x*1’*x*2*x*3, and *x*1*x*2*x*3. Finally, there are two implicants: First implicant that covers a group of four minterms results into x1’ and second implicant that covers group of two minterms results into x2x3



Three-variable function $f(x_1, x_2, x_3) = \sum m(0, 1, 2, 3, 7)$.

Terminology

- **Prime Implicant: An implicant is called a *prime implicant* if it cannot be combined into another implicant that has fewer literals. Another way of stating this definition is to say that it is impossible to delete any literal in a prime implicant and still have a valid implicant.**
- Example: there are two prime implicants: $x1'$ and $x2x3$. It is not possible to delete a literal in either of them.
- **Cover: A collection of implicants that account for all valuations for which a given function is equal to 1 (or 0 in case of POS) is called a *cover* of that function. A number of different covers exist for most functions. A set of all prime implicants is a cover.** A cover defines a particular implementation of the function. $f = x1' + x2x3$ is a cover here in this example.



Three-variable function $f(x_1, x_2, x_3) = \sum m(0, 1, 2, 3, 7)$.

Terminology

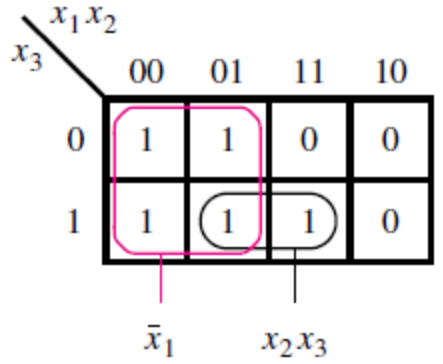
- **Cost: Cost of a logic circuit is the number of gates plus the total number of inputs to all gates in the circuit.** we will assume that primary inputs, namely, the input variables, are available in both true and complemented forms at zero cost.
- Example: The expression $f = x1x2' + x3x4'$ has a cost of 9 because it can be implemented using two AND gates and one OR gate.
- $g = (x1x2' + x3)'(x4' + x5)$ *Cost = 9 input wires + 5 gates = 14*
- In general, the larger the circuit, the more important the cost issue becomes.
- The main objective is to obtain a minimum-cost circuit.

Strategy for Minimization

- **If a prime implicant includes a minterm for which**

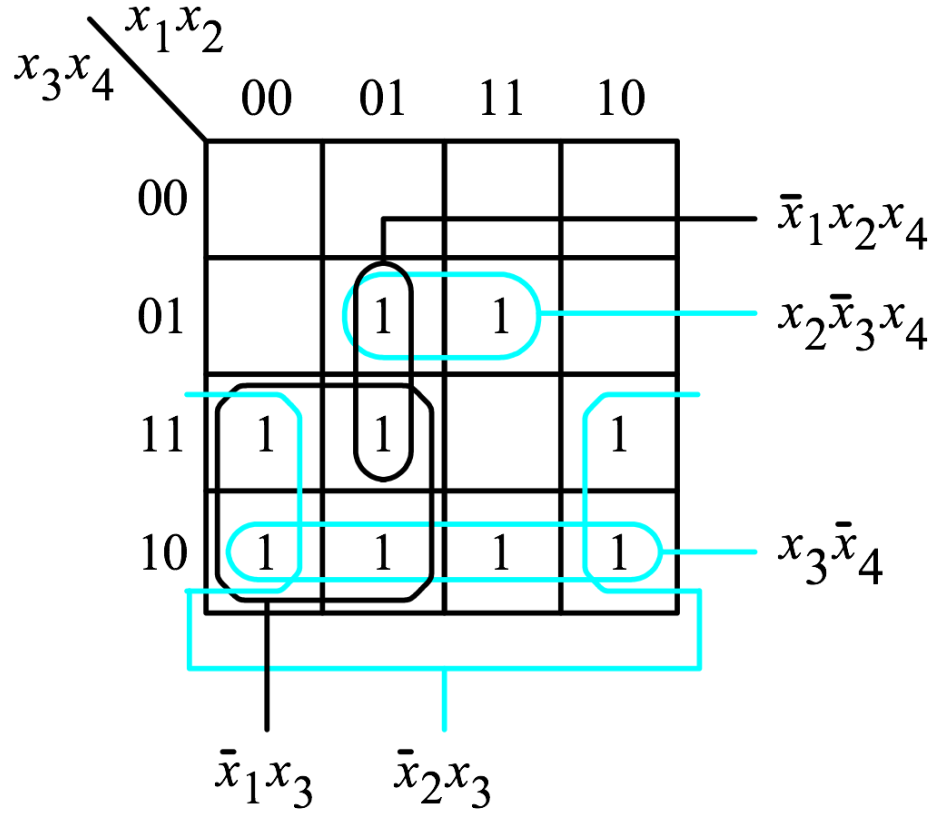
$f = 1$ that is not included in any other prime implicant, then it must be included in the cover and is called an *essential prime implicant.*

- Example: The prime implicants are essential.
- The term $x2x3$ is the only prime implicant that covers the minterm $m7$, and $x1'$ is the only one that covers the minterms $m0$, $m1$, and $m2$. Notice that the minterm $m3$ is covered by both of these prime implicants. The minimum-cost realization of the function is
- $f = x1' + x2x3$



Three-variable function $f(x_1, x_2, x_3) = \sum m(0, 1, 2, 3, 7)$.

Example 1 for Minimization



PI = {x1'x3, x2'x3, x3x4', x2x3'x4, x1'x2x4}
EPI = {x3x4', x2'x3, x2x3'x4}
Cover = {x3x4', x2'x3, x1'x3, x2x3'x4}

$$f = \bar{x}_2x_3 + x_3\bar{x}_4 + x_2\bar{x}_3x_4 + \bar{x}_1x_3$$

Cost = 13 + 5 = 18

Figure 4.10. Four-variable function $f(x_1, \ldots, x_4) = \Sigma\, m(2, 3, 5, 6, 7, 10, 11, 13, 14)$.

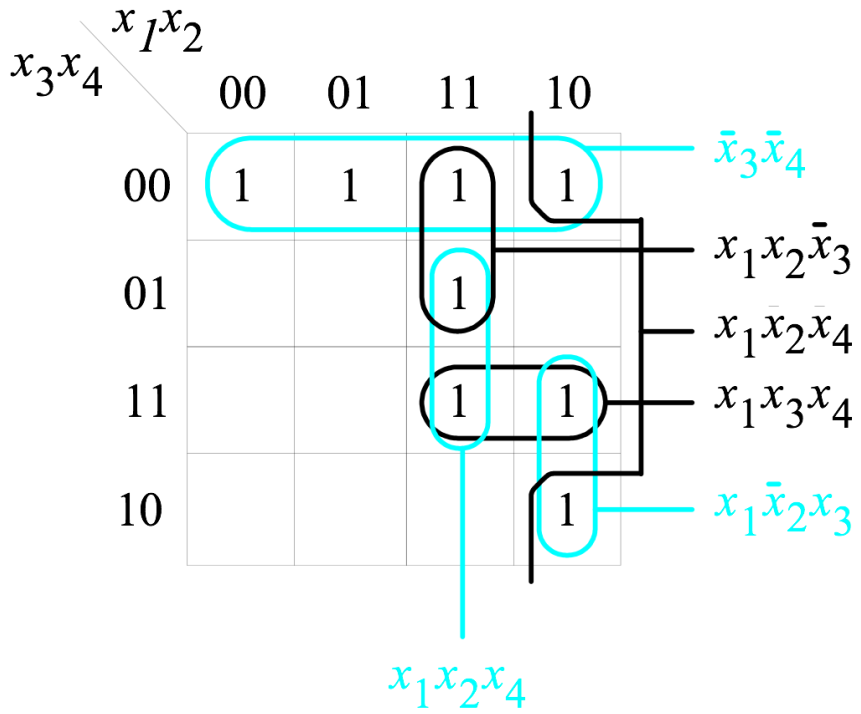
- The process of finding a minimum-cost circuit involves the following steps:

1. Generate all prime implicants for the given function f.

2. Find the set of essential prime implicants.

3. If the set of essential prime implicants covers all valuations for which $f = 1$, then this set is the desired cover of f. Otherwise, determine the nonessential prime implicants that should be added to form a complete minimum-cost cover. The choice of nonessential prime implicants to be included in the cover is governed by the cost considerations.

Example 2



PI = { x3'x4', x1x2x3', x1x2x4, x1x3x4, x1x2'x3, x1x2'x4'}
EPI = {x3'x4'}
Cover 1 = {x3'x4', x1x2x3', x1x3x4, x1x2'x3}
Cover 2 = {x3'x4', x1x2x4,x1x2'x3}

$$f = \bar{x}_3\bar{x}_4 + x_1x_2\bar{x}_3 + x_1x_3x_4 + x_1\bar{x}_2x_3$$

Cost = 15 + 5 = 20

Figure 4.11. The function $f(x_1,\ldots,x_4) = \Sigma\, m(0, 4, 8, 10, 11, 12, 13, 15)$.

Example 3

PI = {x1'x3'x4', x2x3'x4, x1x3x4, x2'x3x4', x1'x2'x4', x1'x2x3', x1x2x4, x1x2'x3}
EPI = {No EPI}
Cover = Either this {x1'x3'x4', x2x3'x4, x1x3x4, x2'x3x4'}
or this {x1'x2'x4', x1'x2x3', x1x2x4, x1x2'x3}

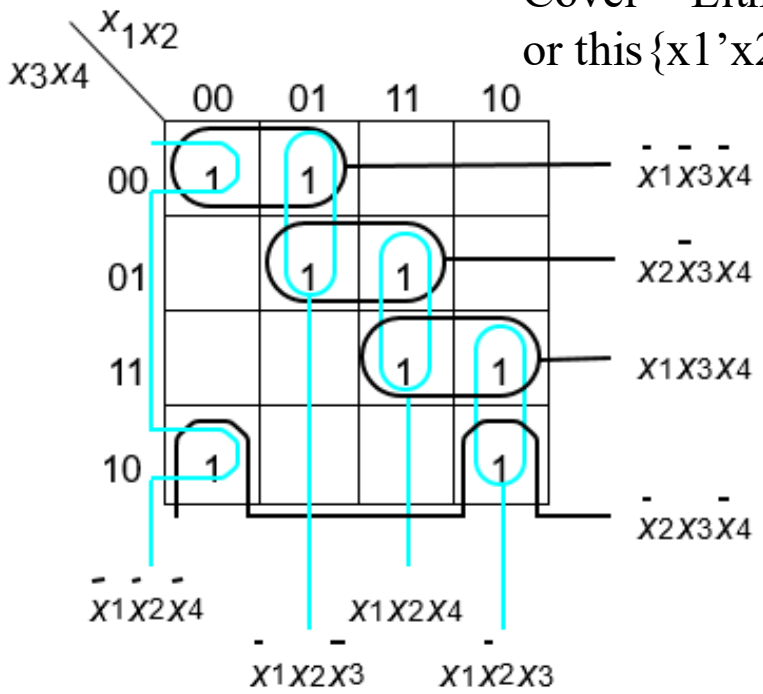


Figure 4.12. The function $f(x_1, \ldots, x_4) = \Sigma\, m(0, 2, 4, 5, 10, 11, 13, 15)$.

Sometimes there may not be any essential prime implicants at all. An example is given in Figure 4.12. Choosing any of the prime implicants and first including it, then excluding it from the cover leads to two alternatives of equal cost. One includes the prime implicants indicated in black, which yields

$$f = \bar{x}_1\bar{x}_3\bar{x}_4 + x_2\bar{x}_3x_4 + x_1x_3x_4 + \bar{x}_2x_3\bar{x}_4$$

The other includes the prime implicants indicated in blue, which yields

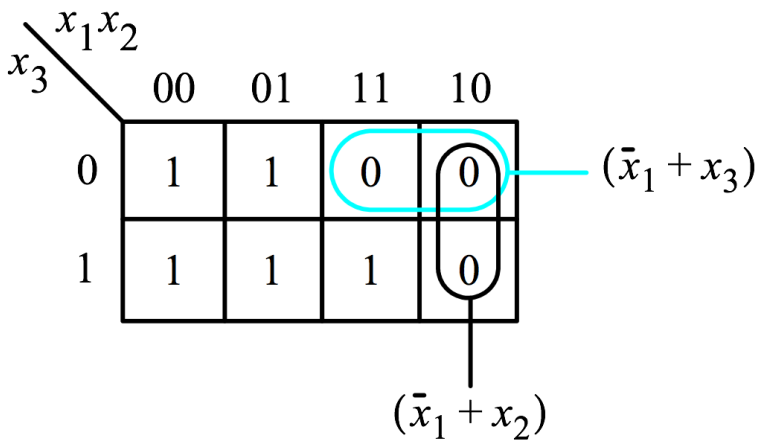
$$f = \bar{x}_1\bar{x}_2\bar{x}_4 + \bar{x}_1x_2\bar{x}_3 + x_1x_2x_4 + x_1\bar{x}_2x_3$$

Minimization of Product-of-Sums Forms

- To find the minimum-cost sum-of-products (SOP) implementations

of functions, we can use the same techniques and the principle of duality to obtain minimum cost product-of-sums (POS) implementations. In this case it is the maxterms for which $f = 0$ that have to be combined into sum terms that are as large as possible. Again, a sum term is considered larger if it covers more maxterms, and the larger the term, the less costly it is to implement.

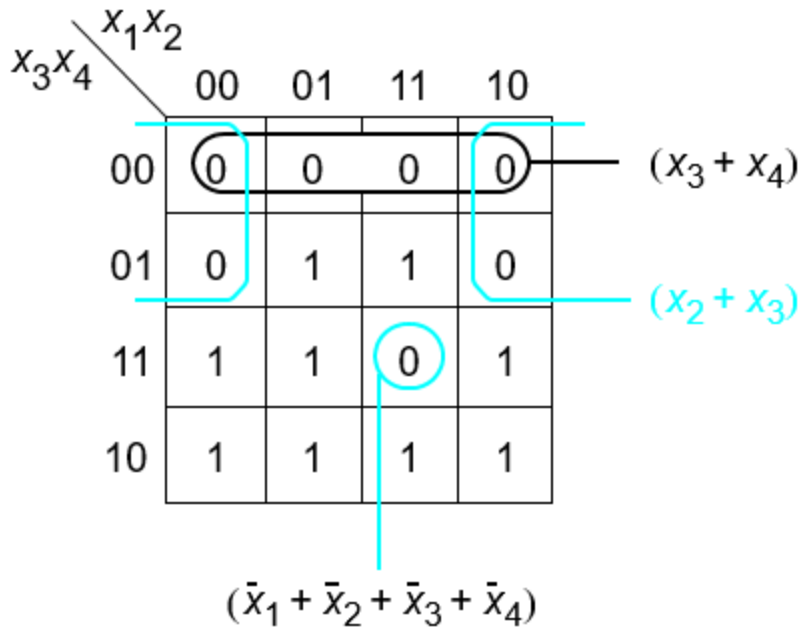
Example 4



There are three maxterms that must be covered: M_4, M_5, and M_6. They can be covered by two sum terms shown in the figure, leading to the following implementation:

$$f = (\bar{x}_1 + x_2)(\bar{x}_1 + x_3)$$

Example 5



$$f = (x_2 + x_3)(x_3 + x_4)(\bar{x}_1 + \bar{x}_2 + \bar{x}_3 + \bar{x}_4)$$

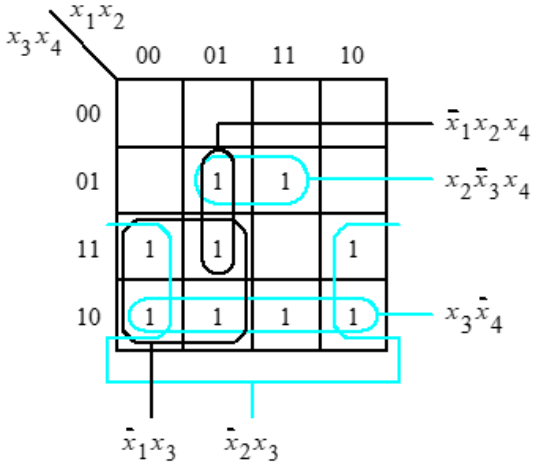
Figure 4.14. POS minimization of $f(x_1,\ldots, x_4) = \Pi M(0, 1, 4, 8, 9, 12, 15)$.

Assuming that both the complemented and uncomplemented versions of the input variables x_1 to x_4 are available at no extra cost, the cost of this circuit in POS is 11+ 4 = 15.

But for the same function in the SOP implementation requires cost = 13 + 5 = 18.

Hence, for the given function, the minimum cost implementation solution is POS

Example 1 for Minimization



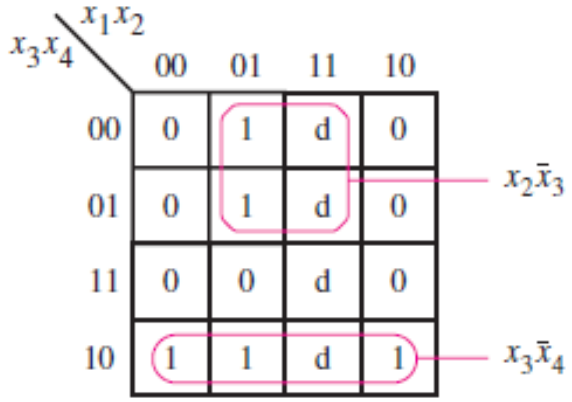
$$f = \bar{x}_2x_3 + x_3\bar{x}_4 + x_2\bar{x}_3x_4 + \bar{x}_1x_3$$

Cost = 13 + 5 = 18

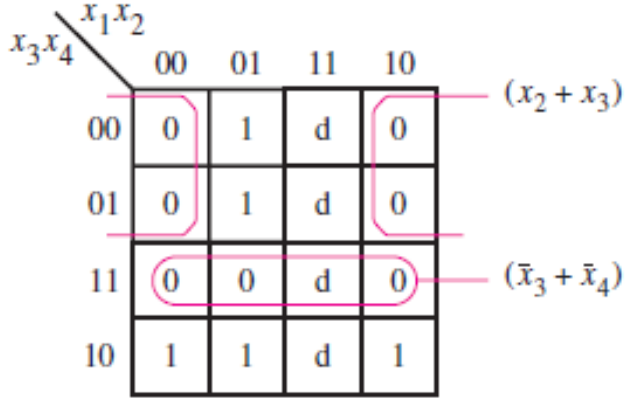
Figure 4.10. Four-variable function $f(x_1,\ldots, x_4) = \Sigma m(2, 3, 5, 6, 7, 10, 11, 13, 14)$.

Incompletely Specified Functions

- In digital systems it often happens that certain input conditions can never occur and is a *don't-care condition*. A function that has don't-care condition(s) is said to be *incompletely specified*. Don't-care conditions, or *don't cares* for short, can be used to advantage in the design of logic circuits. Since these input valuations will never occur, the designer may assume that the function value for these valuations is either 1 or 0, whichever is more useful in trying to find a minimum-cost implementation.
- If there are *k* don't cares, then there are 2^*k (2 to the power k)* possible ways of assigning 0 or 1 values to them.



(a) SOP implementation



(b) POS implementation

Figure 4.15 Two implementations of the function $f(x_1, \ldots, x_4) = \sum m(2, 4, 5, 6, 10) + D(12, 13, 14, 15)$.

$$f(x_1, \ldots, x_4) = \sum m(2, 4, 5, 6, 10) + D(12, 13, 14, 15)$$

where D is the set of don't cares.

Part *(a)* of the figure indicates the best sum-of-products implementation. To form the largest possible groups of 1s, thus generating the lowest-cost prime implicants, it is necessary to assume that the don't cares $D12$, $D13$, and $D14$ (corresponding to minterms $m12$, $m13$, and $m14$) have the value of 1 while $D15$ has the value of 0. Then there are only two prime implicants, which provide a complete cover of f. The resulting implementation is

$$f = x_2\bar{x}_3 + x_3\bar{x}_4$$

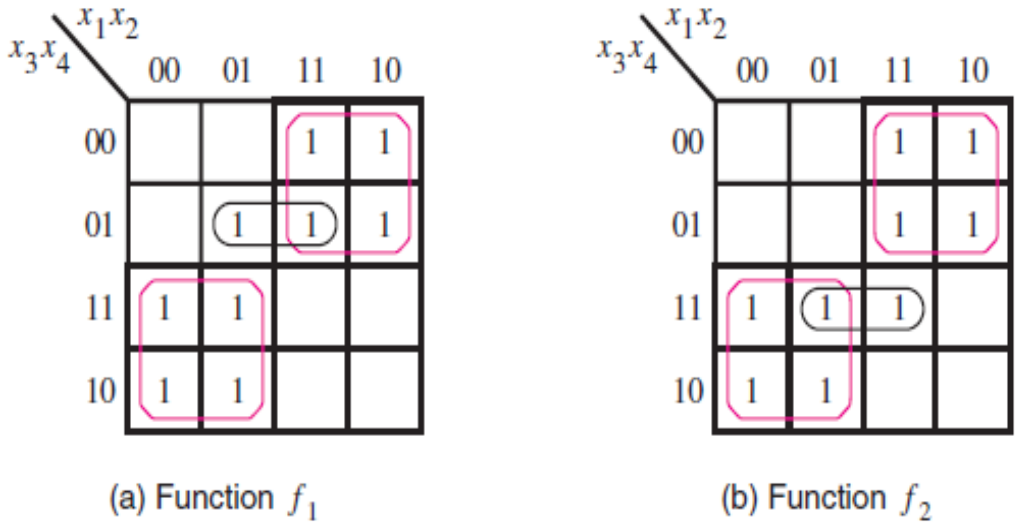
Part *(b)* shows how the best product-of-sums implementation can be obtained. The same values are assumed for the don't cares. The result is

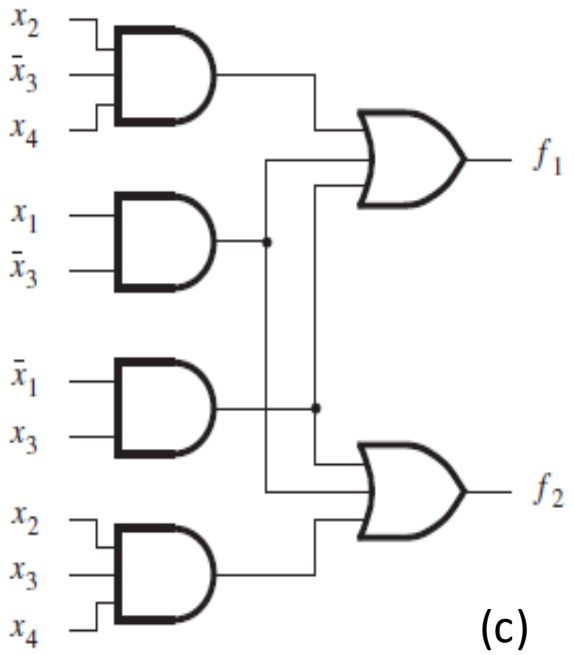
$$f = (x_2 + x_3)(\bar{x}_3 + \bar{x}_4)$$

Multiple-Output Circuits

- In practical digital systems it is necessary to implement a number of functions as part of some large logic circuit. Circuits that implement these functions can often be combined into a less-expensive single circuit with multiple outputs by sharing some of the gates needed in the implementation of individual functions.

Example



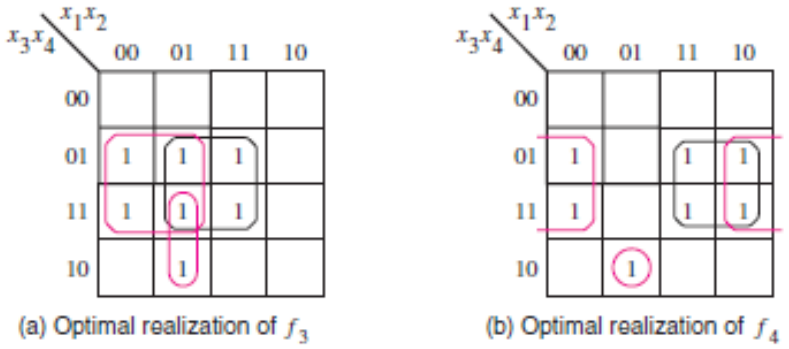


(c)

- The cost of $f1$ = 4 gates + 10 inputs wires = 14.
- The cost of $f2$ is the same.
- Thus, the total cost is 28 if both functions are implemented by separate circuits.
- Low cost realization is possible if the two circuits are combined into a single circuit with two outputs.
- Because the first two product terms are identical in both expressions, the AND gates that implement them need not be duplicated. The combined circuit is shown in (*c*). Its cost is six gates and 16 inputs, for a total of 22.

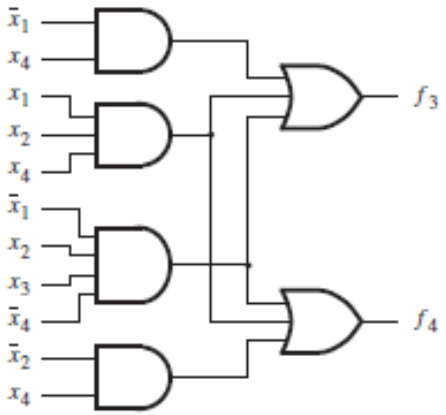
$$f_1 = x_1\bar{x}_3 + \bar{x}_1x_3 + x_2\bar{x}_3x_4$$

$$f_2 = x_1\bar{x}_3 + \bar{x}_1x_3 + x_2x_3x_4$$



$$f_3 = \bar{x}_1x_4 + x_2x_4 + \bar{x}_1x_2x_3$$

$$f_4 = x_1x_4 + \bar{x}_2x_4 + \bar{x}_1x_2x_3\bar{x}_4$$



$$f_3 = x_1x_2x_4 + \bar{x}_1x_2x_3\bar{x}_4 + \bar{x}_1x_4$$

$$f_4 = x_1x_2x_4 + \bar{x}_1x_2x_3\bar{x}_4 + \bar{x}_2x_4$$

- Realizations of the individual functions *f*3 and *f*4 are obtained from parts *(a)* and *(b)*. None of the AND gates can be shared, which means that the cost of the combined circuit = 6 AND gates + 2 OR gates + 21 inputs = 29.
- The combined realization of the functions in (c), shows that the first two implicants are identical in both expressions. The resulting circuit is given in (*d*). It has the cost = 6 gates + 17 inputs = 23.

Problems for Practice - Tutorial

- Repeat the above two examples for POS implementation
- 1. f1 and f2 in POS
- 2. f3 and f4 in POS